MID SEMESTER EXAMINATION

Question Bank with Answers

Deep Learning & Machine Data Mining

7 Marks per Question • Compiled Question Bank

Contents

Module 1

Q1. Explain the fundamentals of Deep Learning.
Q2. Explain Probabilistic Modelling with a suitable example.
Q3. Explain the Perceptron Learning Algorithm with a suitable example.
Q4. Differentiate between Machine Learning and Deep Learning.
Q5. Explain Scalars, Vectors, Matrices and Tensors with examples.
Q6. Explain Tensor Manipulation and its Common Operations.

Module 2

Q1. Explain the basic structure of a Neural Network with a neat diagram.
Q2. Explain the Building Blocks of a Neural Network.
Q3. Explain the different Activation Functions used in Neural Networks.
Q4. Explain Loss Functions and their importance in Neural Networks.
Q5. Explain Overfitting and Underfitting with suitable examples.
Q6. Explain Feature Engineering with a suitable example.

Module 3

Q1. What is a Convolutional Neural Network (CNN)? Explain its basic concept.
Q2. Explain the main applications of Convolutional Neural Networks (CNNs).
Q3. Explain the basic working of a Convolutional Neural Network (CNN) with a suitable example.
Q4. What are LTI (Linear Time-Invariant) Systems? Explain their basic characteristics.
Q5. Explain the concept of Linear Time-Invariant (LTI) systems with a suitable example.
Q6. What is an LTI (Linear Time-Invariant) System? Explain its two main properties.
Q7. Define convolution. Explain how a filter moves across an image to perform convolution.

Module 1

Q1. Explain the fundamentals of Deep Learning. *[7 Marks]*

Deep Learning is a subfield of Machine Learning that uses artificial neural networks with multiple layers to learn complex patterns and representations directly from data.

Fundamentals of Deep Learning:

1. Artificial Neural Networks:

Deep Learning is based on artificial neural networks consisting of interconnected neurons. Each neuron receives inputs, applies weights and produces an output.

2. Multiple Layers:

Deep neural networks contain multiple layers such as input layer, hidden layers and output layer. Multiple hidden layers enable the network to learn complex representations.

3. Representation Learning:

Deep Learning automatically learns useful features and representations from raw data instead of depending completely on manually designed features.

4. Training:

The network learns by comparing its predicted output with the actual output. The error is used to update the weights of the network.

5. Backpropagation:

Backpropagation calculates the error contribution of different weights and propagates the error backward through the network.

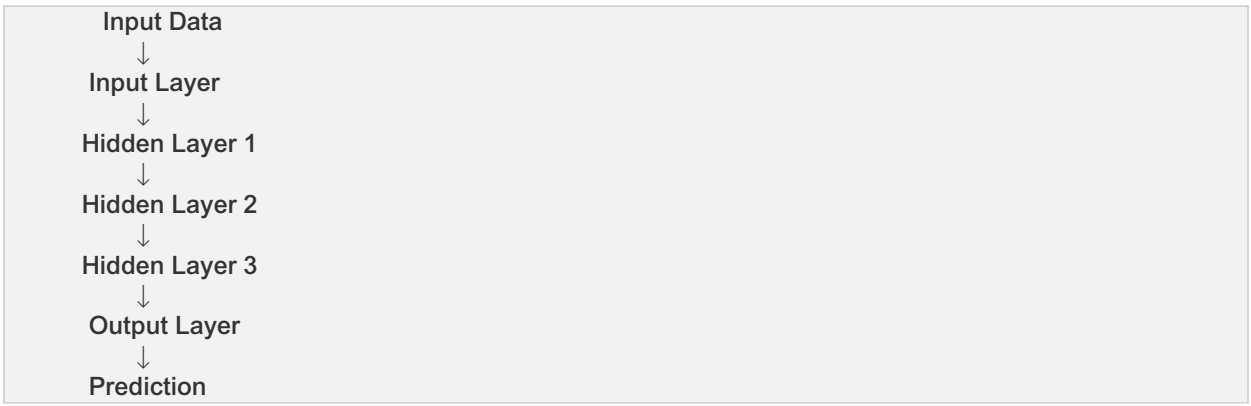
6. Optimization:

Optimization algorithms such as gradient-based methods are used to adjust network weights and minimize the loss function.

7. Large Amount of Data and Computation:

Deep Learning generally benefits from large datasets and high computational power for training complex neural networks.

Basic Structure:



Applications of Deep Learning:

- Image and object recognition
- Speech recognition
- Natural language processing
- Recommendation systems
- Medical image analysis
- Autonomous systems

Thus, Deep Learning uses multilayer neural networks, representation learning, backpropagation and optimization to automatically learn complex patterns from large amounts of data.

Q2. Explain Probabilistic Modelling with a suitable example. *[7 Marks]*

Probabilistic modelling is a mathematical approach that represents uncertainty in data and relationships between variables using probability distributions.

It is useful when the available data is uncertain, incomplete or noisy.

Main Concepts of Probabilistic Modelling:

1. Random Variables:

A random variable represents a quantity whose value can vary according to different possible outcomes.

2. Probability Distribution:

A probability distribution describes the likelihood of different values or outcomes of a random variable.

3. Parameters:

Parameters define the characteristics of a probability distribution and are generally estimated from available data.

4. Observed and Hidden Variables:

Observed variables are directly available in the data, while hidden variables cannot be directly observed and may need to be inferred.

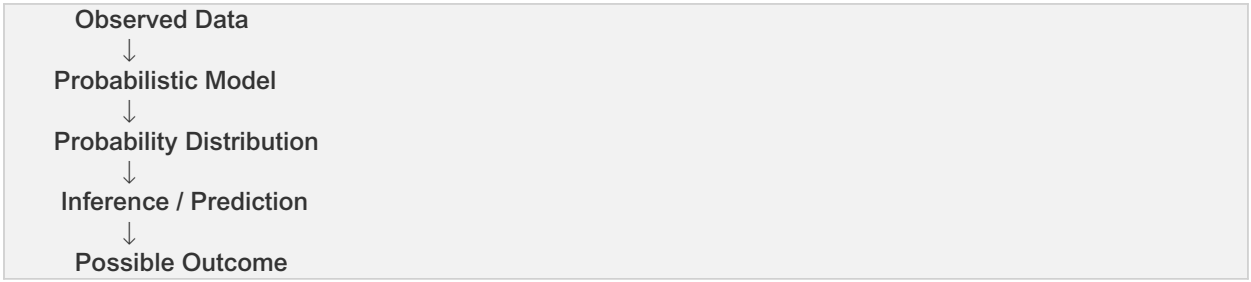
5. Inference:

Inference is the process of estimating unknown variables or probabilities from observed data.

6. Prediction:

The trained probabilistic model can be used to predict the probability of future or unknown outcomes.

General Representation:



Example:

Consider an email classification system. Let an email be classified as Spam or Not Spam.

The model estimates:

P(Spam | Email)

If the probability of an email being spam is high, the email is classified as Spam. If the probability is low, it is classified as Not Spam.

For example:

$$P(Spam \mid Email) = 0.90$$

Therefore, the email has a high probability of being Spam.

Applications:

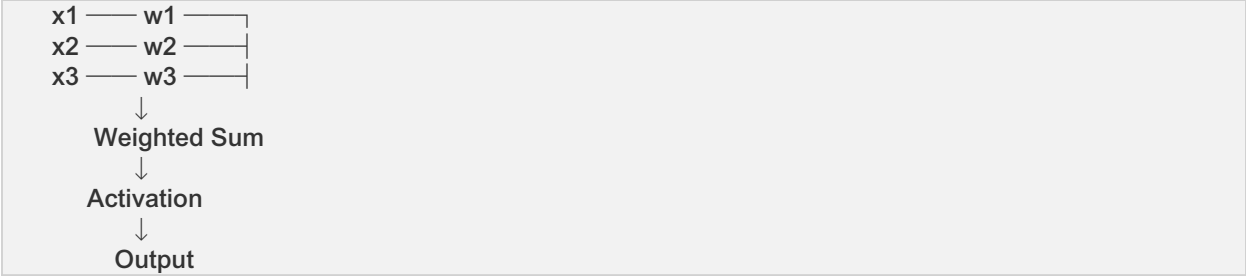
- Classification
- Prediction
- Speech recognition
- Natural language processing
- Medical diagnosis
- Risk analysis

Thus, probabilistic modelling provides a systematic way to represent uncertainty and make predictions using probability.

Q3. Explain the Perceptron Learning Algorithm with a suitable example. *[7 Marks]*

A Perceptron is a basic artificial neuron used for binary classification. It takes input values, multiplies them by corresponding weights, adds a bias and produces an output using an activation function.

Structure of Perceptron:



The weighted sum is:

$$z = w1x1 + w2x2 + ... + wnxn + b$$

where:

x = input w = weight b = bias

For binary classification, a step activation function can be used:

y = 1, if z > 0 y = 0, otherwise

Perceptron Learning Algorithm:

1. Initialize the weights and bias, generally with small values.

2. Select a training input and its target output.

3. Calculate the weighted sum.

4. Apply the activation function to obtain predicted output.

5. Calculate the error.

Error = Target Output - Predicted Output

6. Update the weights using:

$$w_i(new) = w_i(old) + \eta \times Error \times x_i$$

7. Update the bias using:

$$b(new) = b(old) + \eta \times Error$$

8. Repeat the process for the training examples until the classification is correct or the stopping condition is reached.

Example:

For an AND logic gate:

Input x1	Input x2	Target
0	0	0
0	1	0
1	0	0
1	1	1

The perceptron learns suitable weights and bias so that only the input combination (1,1) produces output 1.

Thus, the Perceptron Learning Algorithm trains a simple neural network by iteratively updating weights and bias based on classification error.

Q4. Differentiate between Machine Learning and Deep Learning. *[7 Marks]*

Machine Learning (ML) is a field in which algorithms learn patterns from data to make predictions or decisions. Deep Learning (DL) is a subset of Machine Learning that uses multilayer neural networks to learn complex representations.

Difference between Machine Learning and Deep Learning:

1. Definition:

Machine Learning uses different algorithms to learn from data, whereas Deep Learning uses multilayer neural networks.

2. Feature Extraction:

In traditional Machine Learning, features are often selected or engineered manually. Deep Learning can automatically learn useful features from raw data.

3. Data Requirement:

Machine Learning can often work with comparatively smaller datasets, while Deep Learning generally benefits from large amounts of data.

4. Computational Requirement:

Machine Learning generally requires less computational power. Deep Learning models can require significant computational resources.

5. Training Time:

Machine Learning models generally require less training time. Deep Learning models can require longer training because of their large number of parameters and layers.

6. Model Complexity:

Machine Learning models may use algorithms such as decision trees, support vector machines and regression. Deep Learning mainly uses neural networks with multiple layers.

7. Applications:

Machine Learning is commonly used for prediction, classification and data analysis. Deep Learning is widely used in image recognition, speech recognition and natural language processing.

Comparison:

Machine Learning:

```
Data → Feature Engineering → ML Algorithm → Prediction
```

Deep Learning:

```
Data → Neural Network → Automatic Feature Learning → Prediction
```

Thus, Deep Learning is a specialized approach within Machine Learning that uses multilayer neural networks and automatic representation learning to solve complex problems.

Q5. Explain Scalars, Vectors, Matrices and Tensors with examples. *[7 Marks]*

Scalars, vectors, matrices and tensors are mathematical structures used to represent and process data in Machine Learning and Deep Learning.

1. Scalar:

A scalar is a single numerical value. It has no direction or multiple dimensions.

Example:

$$x = 5$$

A scalar can represent values such as temperature, age or learning rate.

2. Vector:

A vector is an ordered collection of scalar values. It is generally represented as a one-dimensional array.

Example:

$$V = [2, 4, 6]$$

A vector can represent the features of a single data sample.

3. Matrix:

A matrix is a rectangular arrangement of numbers organized into rows and columns. It is a two-dimensional data structure.

Example:

```
    [1  2  3]
A = [4  5  6]
```

The matrix has 2 rows and 3 columns.

4. Tensor:

A tensor is a multidimensional generalization of scalars, vectors and matrices. It can have two or more dimensions.

Examples:

- Scalar → 0-dimensional tensor
- Vector → 1-dimensional tensor
- Matrix → 2-dimensional tensor
- Image → commonly represented as a 3-dimensional tensor

Example of an image tensor:

```
Height × Width × Channels
224 × 224 × 3
```

where 3 represents the Red, Green and Blue colour channels.

Representation:

```
Scalar
  ↓
Vector
  ↓
Matrix
  ↓
Tensor
Increasing Dimensions
```

Applications in Deep Learning:

- Inputs to neural networks
- Image representation
- Text and sequence representation
- Weight storage
- Batch processing

Thus, scalars, vectors, matrices and tensors provide the basic data structures required for representing and processing data in Deep Learning systems.

Q6. Explain Tensor Manipulation and its Common Operations. *[7 Marks]*

Tensor manipulation refers to operations performed on tensors to change their shape, dimensions, arrangement or values while processing data in Deep Learning.

Common Tensor Operations:

1. Reshaping:

Reshaping changes the shape of a tensor without changing its data.

Example:

A tensor of shape (2, 3) can be reshaped into (3, 2).

2. Transpose:

Transpose exchanges the dimensions of a tensor. For a matrix, rows become columns and columns become rows.

3. Slicing:

Slicing extracts a portion of a tensor.

Example:

Selecting particular rows, columns or elements from a tensor.

4. Indexing:

Indexing is used to access a specific element or position of a tensor.

5. Broadcasting:

Broadcasting allows operations between tensors with compatible but different shapes by automatically expanding dimensions.

6. Concatenation:

Concatenation joins two or more tensors along a specified dimension.

7. Stacking:

Stacking combines tensors by creating a new dimension.

8. Element-wise Operations:

Mathematical operations such as addition, subtraction, multiplication and division can be performed element by element on tensors.

Example:

$$A = [1\ 2] \quad [3\ 4]$$

$$B = [5\ 6] \quad [7\ 8]$$

Element-wise addition:

$$A + B = [6\ \ 8] \qquad [10\ 12]$$

Common Tensor Manipulations:



Tensor manipulation is important because Deep Learning models require data to be arranged into appropriate shapes and dimensions before performing mathematical operations.

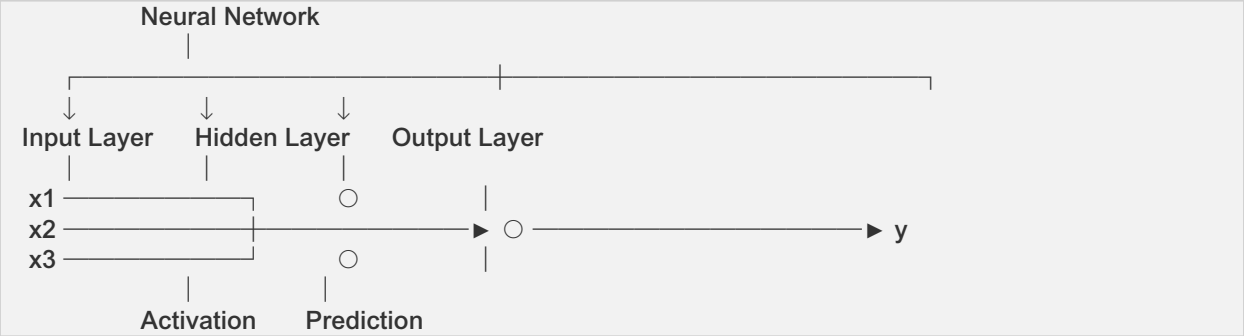
Thus, tensor manipulation provides the necessary operations for organizing, transforming and processing multidimensional data in Deep Learning.

Module 2

Q1. Explain the basic structure of a Neural Network with a neat diagram. *[7 Marks]*

A Neural Network is a computational model inspired by the working of the human brain. It consists of interconnected processing units called neurons that learn patterns from input data.

Basic Structure of a Neural Network:



1. Input Layer:

The input layer receives the input features or data. Each input neuron represents one feature of the data.

2. Hidden Layer:

Hidden layers perform computations on the input data. A neural network can contain one or more hidden layers. Deep neural networks contain multiple hidden layers.

3. Output Layer:

The output layer produces the final prediction or result. The number and type of output neurons depend on the problem.

4. Weights:

Each connection between neurons has a weight. Weights determine the importance of the corresponding input.

5. Bias:

Bias is an additional parameter added to the weighted sum. It helps the neuron adjust its output.

6. Activation Function:

The activation function is applied to the weighted sum to introduce non-linearity into the neural network.

The output of a neuron can be represented as:

$$z = w1x1 + w2x2 + ... + wnxn + b$$

Output = f(z)

where x represents inputs, w represents weights, b represents bias and f represents the activation function.

Thus, a neural network receives input data, processes it through weighted connections and hidden layers, and produces the required output through the output layer.

Q2. Explain the Building Blocks of a Neural Network. *[7 Marks]*

A Neural Network is made up of several fundamental components that work together to learn patterns from data and produce predictions.

The main building blocks of a Neural Network are:

1. Neurons:

A neuron is the basic processing unit of a neural network. It receives inputs, performs a weighted sum and produces an output.

2. Weights:

Weights represent the importance of inputs connected to a neuron. During training, the weights are adjusted to reduce prediction error.

3. Bias:

Bias is an additional value added to the weighted sum. It allows the neuron to shift the activation function and improves the flexibility of the model.

4. Activation Function:

An activation function determines the output of a neuron from its weighted input. It introduces non-linearity into the network.

5. Layers:

Neural networks are organized into layers:

- Input layer receives data.
- Hidden layers perform intermediate computations.
- Output layer produces the final result.

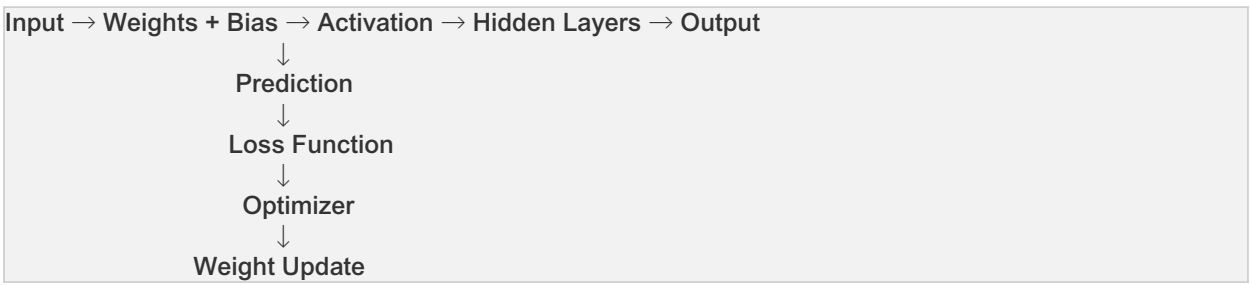
6. Loss Function:

The loss function measures the difference between the predicted output and the actual output. It indicates how well the model is performing.

7. Optimizer:

An optimizer updates the weights and biases of the network to minimize the loss function during training.

Basic Structure:



The major building blocks therefore work together during training to convert input data into accurate predictions.

Thus, neurons, weights, biases, activation functions, layers, loss functions and optimizers form the fundamental building blocks of a neural network.

Q3. Explain the different Activation Functions used in Neural Networks. *[7 Marks]*

An activation function is a mathematical function applied to the output of a neuron. It determines whether and to what extent a neuron should be activated. Activation functions introduce non-linearity into a neural network, allowing it to learn complex patterns.

The commonly used activation functions are:

1. Sigmoid Function:

The sigmoid function maps the input value into the range 0 to 1.

$$f(x) = \frac{1}{1 + e^{(-x)}}$$

It is commonly used for binary classification output layers.

2. Tanh Function:

Tanh maps the input into the range -1 to +1.

$$f(x) = \tanh(x)$$

It is zero-centered and can be useful in certain neural network architectures.

3. ReLU (Rectified Linear Unit):

ReLU produces zero for negative values and the input value for positive values.

$$f(x) = \max(0, x)$$

It is widely used in hidden layers because it is simple and computationally efficient.

4. Leaky ReLU:

Leaky ReLU allows a small negative output for negative input values.

$$f(x) = \begin{cases} x, & \text{if } x > 0 \\ \alpha x, & \text{if } x \leq 0 \end{cases}$$

where α is a small positive value.

5. Softmax:

Softmax converts a set of values into probabilities whose sum is 1. It is commonly used in the output layer for multi-class classification.

Comparison:

Sigmoid	→ Range 0 to 1	→ Binary classification
Tanh	→ Range -1 to +1	→ Hidden layers
ReLU	→ 0 to ∞	→ Common hidden-layer function
Leaky ReLU	→ Negative + positive	→ Helps avoid inactive neurons
Softmax	→ Probabilities	→ Multi-class classification

Thus, activation functions provide non-linearity and enable neural networks to learn complex relationships between input and output data.

Q4. Explain Loss Functions and their importance in Neural Networks. *[7 Marks]*

A Loss Function is a mathematical function that measures the difference between the predicted output of a neural network and the actual target output.

The value of the loss indicates how incorrect the prediction is. A lower loss generally indicates that the model's predictions are closer to the actual values.

Importance of Loss Function:

1. Measures Prediction Error:

It measures the difference between predicted and actual output.

2. Guides Learning:

The loss value provides a basis for updating the model parameters during training.

3. Weight Optimization:

Optimization algorithms use the loss to adjust weights and biases.

4. Model Evaluation:

Loss can be used to evaluate how well the neural network is performing.

Common Loss Functions:

1. Mean Squared Error (MSE):

MSE is commonly used for regression problems.

$$MSE = \frac{1}{n} \sum (y - \hat{y})^2$$

where y is the actual value and ŷ is the predicted value.

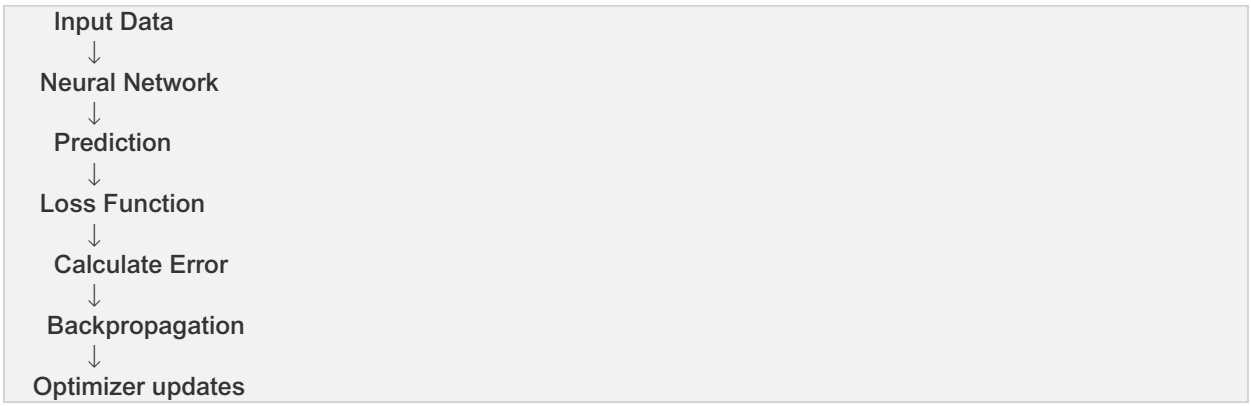
2. Binary Cross-Entropy:

It is commonly used for binary classification problems.

3. Categorical Cross-Entropy:

It is commonly used for multi-class classification problems.

Training Process:



```
weights and bias
      ↓
Improved Model
```

Thus, the loss function is a fundamental component of neural network training because it measures prediction error and guides the optimization process toward better performance.

Q5. Explain Overfitting and Underfitting with suitable examples. *[7 Marks]*

Overfitting and underfitting are common problems that affect the performance of machine learning and neural network models.

1. Overfitting:

Overfitting occurs when a model learns the training data too closely, including noise and unnecessary details. As a result, the model performs very well on training data but performs poorly on unseen or test data.

Example:

Suppose a neural network is trained to classify images of cats and dogs. If the model memorizes the training images instead of learning general features, it may classify training images correctly but make errors on new images.

Characteristics:

- Very low training error
- High validation/test error
- Model is too complex
- Poor generalization

Methods to reduce overfitting:

- Use more training data
- Regularization
- Dropout
- Data augmentation
- Early stopping
- Reduce model complexity

2. Underfitting:

Underfitting occurs when a model is too simple to learn the important patterns in the training data. Therefore, it performs poorly on both training and unseen data.

Example:

If a complex image classification problem is solved using an extremely simple model with insufficient layers and parameters, the model may fail to learn important features.

Characteristics:

- High training error
- High validation/test error
- Model is too simple

- Important patterns are not learned

Comparison:

Overfitting:

```
Training performance → High
Testing performance  → Low
Cause → Model learns noise and specific details
```

Underfitting:

```
Training performance → Low
Testing performance  → Low
Cause → Model fails to learn important patterns
```

Ideal Model:

```
Training Error  → Low
Testing Error   → Low
Generalization → Good
```

Thus, a good neural network should achieve a balance between learning the training data and generalizing well to unseen data.

Q6. Explain Feature Engineering with a suitable example. *[7 Marks]*

Feature Engineering is the process of selecting, transforming, creating and preparing useful input features from raw data so that a machine learning or neural network model can learn effectively.

The quality of input features can significantly affect the performance of a learning model.

Steps in Feature Engineering:

1. Data Understanding:

Understand the available data and identify relevant variables.

2. Feature Selection:

Select the most useful features and remove irrelevant or redundant features.

3. Feature Transformation:

Transform features into a suitable form using techniques such as normalization or standardization.

4. Feature Creation:

Create new useful features from existing data.

5. Categorical Encoding:

Convert categorical values into numerical representations.

6. Feature Scaling:

Bring numerical features into suitable ranges when required.

Example:

Consider a dataset containing:

Age, Height, Weight

Instead of using only Height and Weight, a useful new feature can be created as Body Mass Index (BMI):

$$BMI = \frac{Weight}{Height \times Height}$$

The newly created BMI feature may provide useful information for a prediction model.

Example Process:

```
Raw Data
   ↓
Select Relevant Features
   ↓
Clean / Transform Data
   ↓
Create New Features
   ↓
Encode Categorical Data
   ↓
Scale Features
   ↓
Machine Learning / Deep Learning Model
```

Benefits:

- Improves model performance
- Removes irrelevant information
- Makes patterns easier to learn
- Reduces unnecessary features
- Improves prediction accuracy

Thus, feature engineering converts raw data into meaningful and useful features that help a learning model perform better.

Module 3

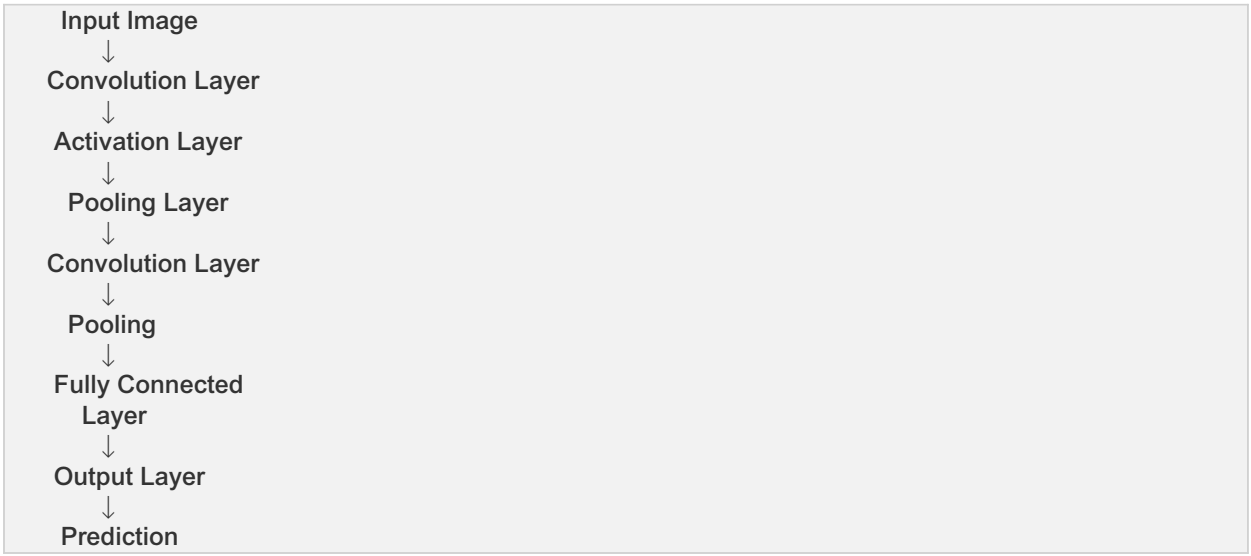
Q1. What is a Convolutional Neural Network (CNN)? Explain its basic concept. *[7 Marks]*

A Convolutional Neural Network (CNN) is a type of deep neural network mainly used for processing data having a grid-like structure, especially images. CNN automatically learns important features such as edges, textures, shapes and objects from input images.

Basic Concept of CNN:

A CNN applies filters or kernels to the input image. The filters move across the image and perform convolution operations to produce feature maps. These feature maps are then processed through activation and pooling layers to extract important features.

Basic Architecture:



Main Components:

1. Input Layer:

It receives the input image in the form of numerical pixel values.

2. Convolution Layer:

Filters are applied to the input to extract important features such as edges, corners and textures.

3. Activation Function:

A nonlinear activation function such as ReLU is applied to the feature maps.

4. Pooling Layer:

Pooling reduces the spatial dimensions of feature maps while retaining important information.

5. Fully Connected Layer:

The extracted features are converted into a form suitable for classification or prediction.

6. Output Layer:

It produces the final classification or prediction.

Advantages of CNN:

- Automatic feature extraction
- Fewer parameters compared with fully connected networks for images
- Effective for image and visual data
- Translation-related feature detection
- High performance in image classification

Applications:

- Image classification
- Object detection
- Face recognition
- Medical image analysis
- Handwritten digit recognition

Thus, CNN is an important deep learning architecture that automatically extracts hierarchical features from images using convolution, activation, pooling and fully connected layers.

Q2. Explain the main applications of Convolutional Neural Networks (CNNs). *[7 Marks]*

Convolutional Neural Networks (CNNs) are deep learning models specially designed to process image and other grid-like data. They automatically learn useful features from input data and are therefore widely used in computer vision and related applications.

Main Applications of CNNs:

1. Image Classification:

CNNs classify images into different categories by automatically learning features such as edges, shapes and textures.

Example: Classifying an image as a cat, dog or bird.

2. Object Detection:

CNNs identify objects present in an image and determine their locations using bounding regions.

Example: Detecting cars and pedestrians in a road image.

3. Face Recognition:

CNNs learn facial features and can be used to recognize or verify faces.

4. Medical Image Analysis:

CNNs are used to analyze medical images such as X-rays, CT scans and MRI images to assist in detecting abnormalities.

5. Handwritten Digit Recognition:

CNNs can recognize handwritten numbers by learning visual patterns from images.

6. Image Segmentation:

CNN-based models can divide an image into meaningful regions or pixels belonging to different objects.

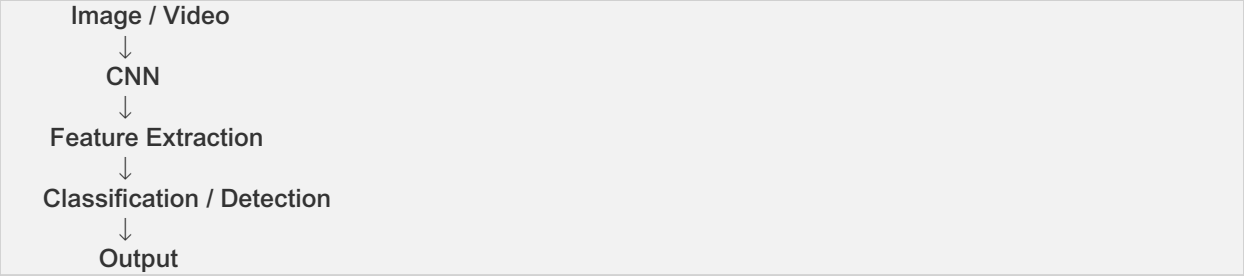
7. Video and Surveillance:

CNNs can extract visual features from video frames and are used in surveillance, activity recognition and monitoring systems.

8. Autonomous Vehicles:

CNNs can process camera images to identify roads, vehicles, pedestrians, traffic signs and other objects.

Basic Application Flow:



Thus, CNNs are widely used in image classification, object detection, face recognition, medical image analysis, segmentation, surveillance and autonomous systems because of their ability to automatically learn important visual features.

Q3. Explain the basic working of a Convolutional Neural Network (CNN) with a suitable example. *[7 Marks]*

A Convolutional Neural Network (CNN) processes an input image by automatically extracting important features and using them to perform classification or prediction.

Basic Working of CNN:

1. Input Image:

The CNN receives an image as input. The image is represented numerically using pixel values.

2. Convolution:

A small filter or kernel moves across the input image. At each position, multiplication and addition operations are performed to generate a feature value.

3. Feature Map:

The convolution operation produces a feature map containing important patterns detected by the filter.

4. Activation:

An activation function such as ReLU is applied to introduce non-linearity.

5. Pooling:

Pooling reduces the spatial dimensions of the feature map while retaining important information. Max pooling is a commonly used technique.

6. Repeated Feature Extraction:

Multiple convolution and pooling layers can be used to learn increasingly complex features.

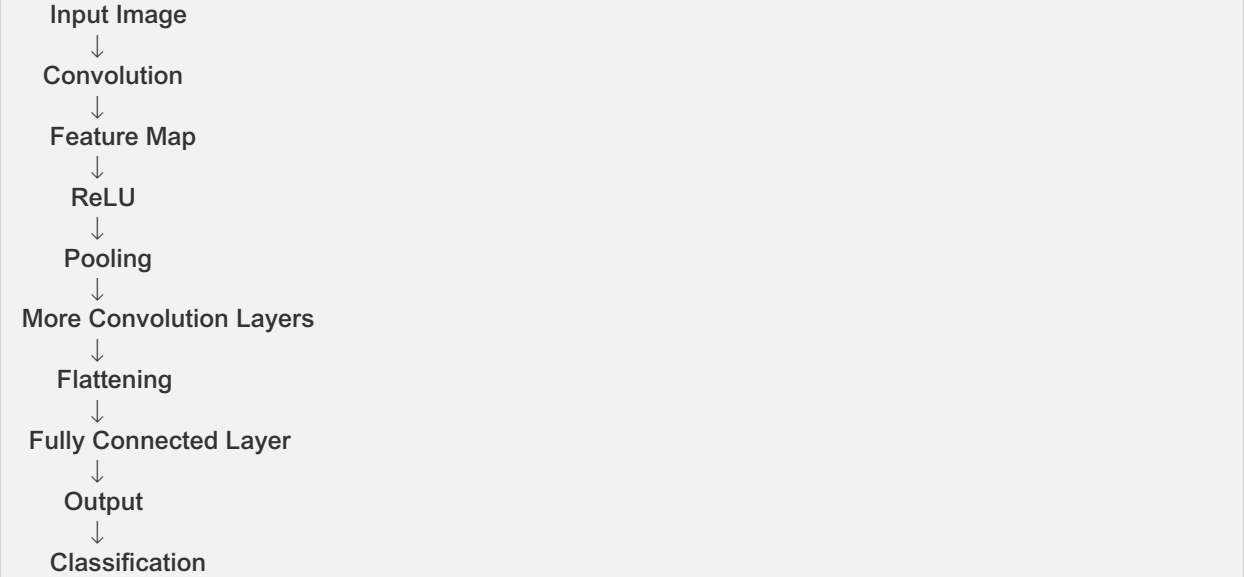
7. Fully Connected Layer:

The extracted features are flattened and passed to fully connected layers for classification.

8. Output:

The output layer produces the final prediction.

Working Diagram:



Example:

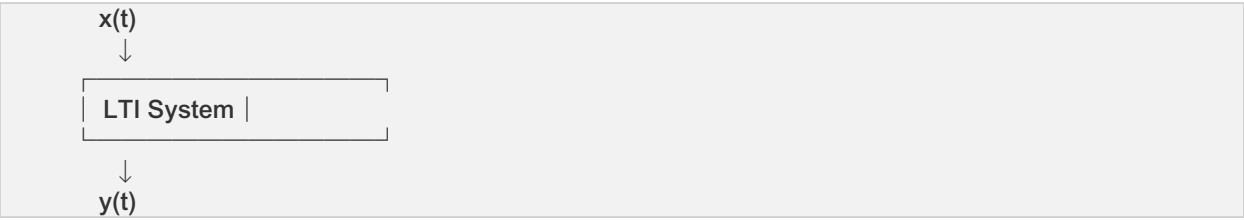
Consider a CNN trained to identify whether an image contains a cat or dog. In the initial convolution layers, the CNN may learn simple features such as edges and corners. Deeper layers combine these features to learn shapes and important parts of the animal. Finally, the fully connected and output layers classify the image as Cat or Dog.

Thus, a CNN works by progressively extracting low-level and high-level features from an image and using these features for the final prediction.

Q4. What are LTI (Linear Time-Invariant) Systems? Explain their basic characteristics. *[7 Marks]*

LTI stands for Linear Time-Invariant. An LTI system is a system that satisfies two fundamental properties: linearity and time invariance. LTI systems are widely used in signal processing and convolution-based analysis.

An LTI system can be represented as:



The two main characteristics of an LTI system are:

1. Linearity:

A system is linear if it satisfies both additivity and homogeneity.

If:

```
x1(t) → y1(t)
x2(t) → y2(t)
```

then for constants a and b:

```
ax1(t) + bx2(t) → ay1(t) + by2(t)
```

This property means that the response to a combination of inputs is equal to the same combination of their individual responses.

2. Time Invariance:

A system is time-invariant if delaying the input by a certain amount causes the output to be delayed by the same amount.

If:

```
x(t) → y(t)
```

then:

```
x(t - t0) → y(t - t0)
```

where t0 is the time shift.

Example:

Consider a system:

$$y(t) = 2x(t)$$

If the input is shifted:

x(t - t0)

the output becomes:

$$y(t) = 2x(t - t0)$$

which is the same as shifting the original output. Therefore, the system is time-invariant and linear.

Importance:

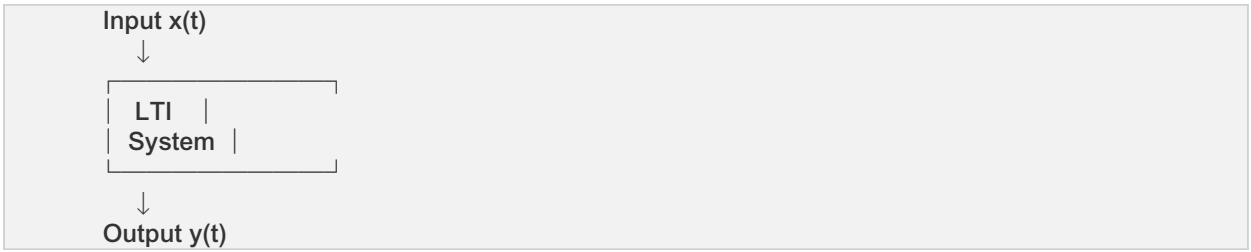
- Used in signal and image processing
- Makes system analysis easier
- Convolution can be used to determine system output
- Provides a mathematical model for many practical systems

Thus, an LTI system is a system that satisfies both linearity and time invariance properties.

Q5. Explain the concept of Linear Time-Invariant (LTI) systems with a suitable example. *[7 Marks]*

A Linear Time-Invariant (LTI) system is a system that satisfies the properties of linearity and time invariance. Such systems are important in signal processing because their output can be determined using the input signal and the system's impulse response.

An LTI system is represented as:



1. Linearity:

A system is linear when it satisfies superposition.

If:

```
x1(t) → y1(t)
x2(t) → y2(t)
```

then:

```
a x1(t) + b x2(t) → a y1(t) + b y2(t)
```

for constants a and b.

2. Time Invariance:

A system is time-invariant if a shift in the input produces the same shift in the output.

If:

```
x(t) → y(t)
```

then:

```
x(t - t0) → y(t - t0)
```

Example:

Consider:

$$y(t) = 3x(t)$$

For an input x(t), the output is:

$$y(t) = 3x(t)$$

Now shift the input by t0:

x(t - t0)

The output becomes:

$$y(t) = 3x(t - t0)$$

This is equal to the shifted original output:

$$y(t - t0) = 3x(t - t0)$$

Therefore, the system is time-invariant. It is also linear because it satisfies the superposition property.

Thus, an LTI system is a system satisfying linearity and time invariance, and it can be effectively analyzed using mathematical tools such as convolution.

Q6. What is an LTI (Linear Time-Invariant) System? Explain its two main properties. *[7 Marks]*

An LTI (Linear Time-Invariant) system is a system that satisfies two fundamental properties: linearity and time invariance. These properties make LTI systems useful for analyzing signals and systems.

The two main properties are:

1. Linearity:

A system is linear if it satisfies the principle of superposition. It includes additivity and homogeneity.

If:

```
x1(t) → y1(t)
x2(t) → y2(t)
```

then for constants a and b:

```
a x1(t) + b x2(t) → a y1(t) + b y2(t)
```

Therefore, the response to a linear combination of inputs is the same linear combination of the corresponding outputs.

2. Time Invariance:

A system is time-invariant if shifting the input signal by a particular amount results in the output being shifted by the same amount.

If:

```
x(t) → y(t)
```

then:

```
x(t - t0) → y(t - t0)
```

where t0 represents the time shift.

Example:

Consider:

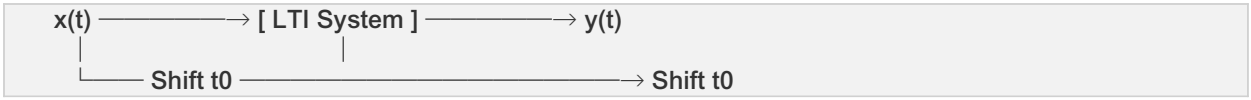
$$y(t) = 2x(t)$$

For shifted input x(t - t0):

$$y(t) = 2x(t - t0)$$

This is equal to shifting the original output by t0. Hence, the system is time-invariant. It is also linear.

Diagram:



Thus, the two essential properties of an LTI system are linearity and time invariance.

Q7. Define convolution. Explain how a filter moves across an image to perform convolution. *[7 Marks]*

Convolution is a mathematical operation used in image processing and Convolutional Neural Networks (CNNs) to extract useful features from an image.

In CNNs, convolution is performed by moving a small matrix called a filter or kernel across the input image.

Basic Process:

1. Input Image:

The image is represented as a matrix of pixel values.

2. Filter/Kernel:

A small matrix containing numerical values is used to detect a particular feature.

3. Sliding:

The filter moves across the image from left to right and top to bottom according to a specified stride.

4. Element-wise Multiplication:

At each position, the filter values are multiplied with the corresponding image pixel values.

5. Summation:

The multiplied values are added to produce one output value.

6. Feature Map:

Repeating this process produces a feature map that represents the features detected by the filter.

Example:

Input image:

```
1 2 3
4 5 6
7 8 9
```

Filter:

```
1 0
0 1
```

For the first position:

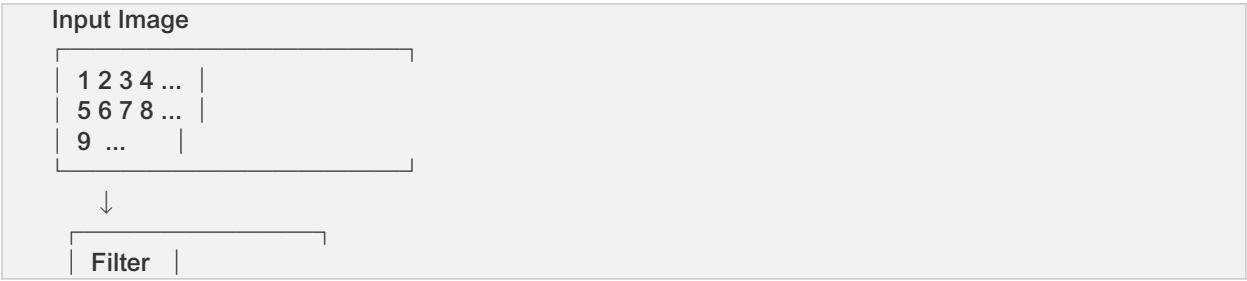
```
1 2
4 5
```

Element-wise multiplication:

```
(1×1) + (2×0) + (4×0) + (5×1)
= 1 + 0 + 0 + 5
= 6
```

The filter then moves to the next position and the same process is repeated.

Diagram:





Important Parameters:

- Kernel/Filter: Detects specific features.
- Stride: Number of pixels by which the filter moves.
- Padding: Extra pixels added around the image.
- Feature Map: Output generated after convolution.

Thus, convolution extracts important local features from an image by sliding a filter across the image and performing element-wise multiplication and summation at each position.